Release 1 : Prise de Rendez-vous

Authentification & Gestion des Rendez-vous

MediConnect — Plateforme e-Santé

NestJS + Angular 17 • PFE 2026

2025 – 2026

3.1 Introduction

Ce chapitre se concentre sur les étapes de réalisation de notre deuxième release en mettant l'accent sur la création et le raffinement du backlog sprint 0 et sprint 1, la conception et la réalisation.

Le sprint 0 couvre les fonctionnalités d'authentification et d'inscription pour l'ensemble des acteurs de la plateforme e-Santé (patients, médecins, pharmaciens et administrateur). Le sprint 1 se concentre sur la prise de rendez-vous médicaux, incluant la recherche de médecins, la sélection de créneaux en temps réel, les confirmations et rappels automatiques.

3.2 Backlog de Release 1

Pour planifier les tâches à accomplir, nous allons créer le backlog du release 2 présenté sur le tableau 3.1. L'estimation est utilisée pour évaluer la charge de travail associée à chaque tâche.

User Story (Backlog Produit)	Priorité	Évaluation	Sprint	Acteur	Workflow
Sprint 1 — *Authentification & Inscription*					
En tant qu'administrateur, je veux m'authentifier pour accéder au back-office	1	Moyen	Sprint 0	Admin	Auth
En tant que patient, je veux m'inscrire (nom, email, mot de passe, profil médical de base)	1	Moyen	Sprint 0	Patient	Auth
En tant que patient, je veux m'authentifier	1	Moyen	Sprint 0	Patient	Auth
En tant que patient, je veux réinitialiser mon mot de passe par email	1	Faible	Sprint 0	Patient	Auth
En tant que médecin, je veux m'inscrire (RPPS, spécialité, cabinet)	1	Moyen	Sprint 0	Médecin	Auth
En tant que médecin, je veux m'authentifier	1	Moyen	Sprint 0	Médecin	Auth
En tant que médecin, je veux réinitialiser mon mot de passe	1	Faible	Sprint 0	Médecin	Auth
En tant que pharmacien, je veux m'inscrire (numéro ordre, officine)	1	Moyen	Sprint 0	Pharmacien	Auth
En tant que pharmacien, je veux m'authentifier	1	Moyen	Sprint 0	Pharmacien	Auth
Sprint 2 — *W1 — Prise de Rendez-vous*					
En tant que patient, je veux rechercher un médecin par spécialité, localisation et disponibilité	2	Fort	Sprint 1	Patient	W1 RDV
En tant que patient, je veux consulter le profil d'un médecin (photo, avis, tarifs)	2	Moyen	Sprint 1	Patient	W1 RDV
En tant que patient, je veux sélectionner un créneau disponible en temps réel (physique ou téléconsultation)	2	Fort	Sprint 1	Patient	W1 RDV
En tant que médecin, je veux recevoir une notification instantanée lors d'une demande de RDV	2	Fort	Sprint 1	Médecin	W1 RDV

User Story (Backlog Produit)	Priorité	Évaluation	Sprint	Acteur	Workflow
En tant que médecin, je veux accepter ou refuser une demande de RDV en 1 clic (statut VALIDÉ / REFUSÉ)	2	Fort	Sprint 1	Médecin	W1 RDV
En tant que patient, je veux recevoir une confirmation de RDV par email et SMS	2	Moyen	Sprint 1	Patient	W1 RDV
En tant que patient, je veux recevoir un rappel automatique 24h avant le RDV avec le lien de téléconsultation	2	Moyen	Sprint 1	Patient	W1 RDV
En tant que patient, je veux annuler ou reporter un RDV depuis mon espace	2	Moyen	Sprint 1	Patient	W1 RDV

Tableau 3.1 – Backlog de Release 1

3.3 Raffinement de Release 1

Dans cette section nous examinons ces cas d'utilisation :

- S'inscrire
- S'authentifier
- Prendre un rendez-vous
- Gérer les rendez-vous

3.3.1 Raffinement de Sprint 1

Raffinement du cas "S'inscrire"

La figure 3.1 offre une représentation visuelle du cas "S'inscrire", décrivant le processus que les utilisateurs suivent pour s'enregistrer sur la plateforme e-Santé :

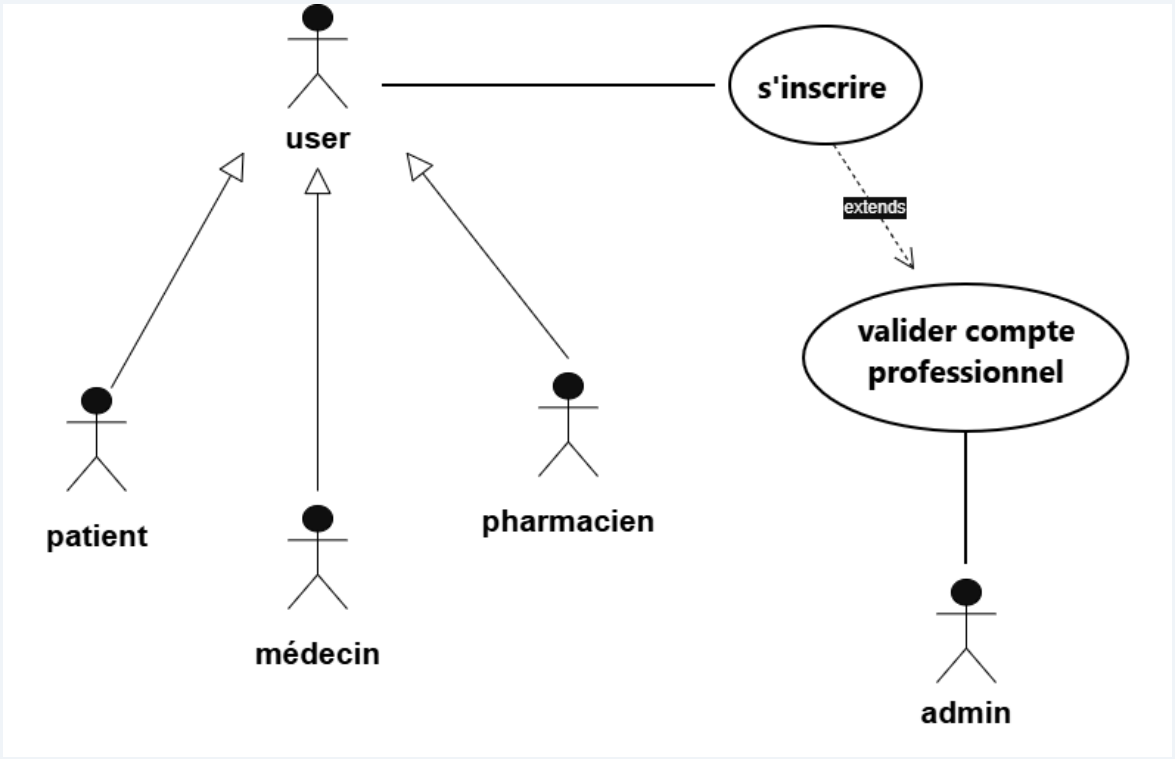


Figure 3.1 – Décomposition de cas d'utilisation « S'inscrire »

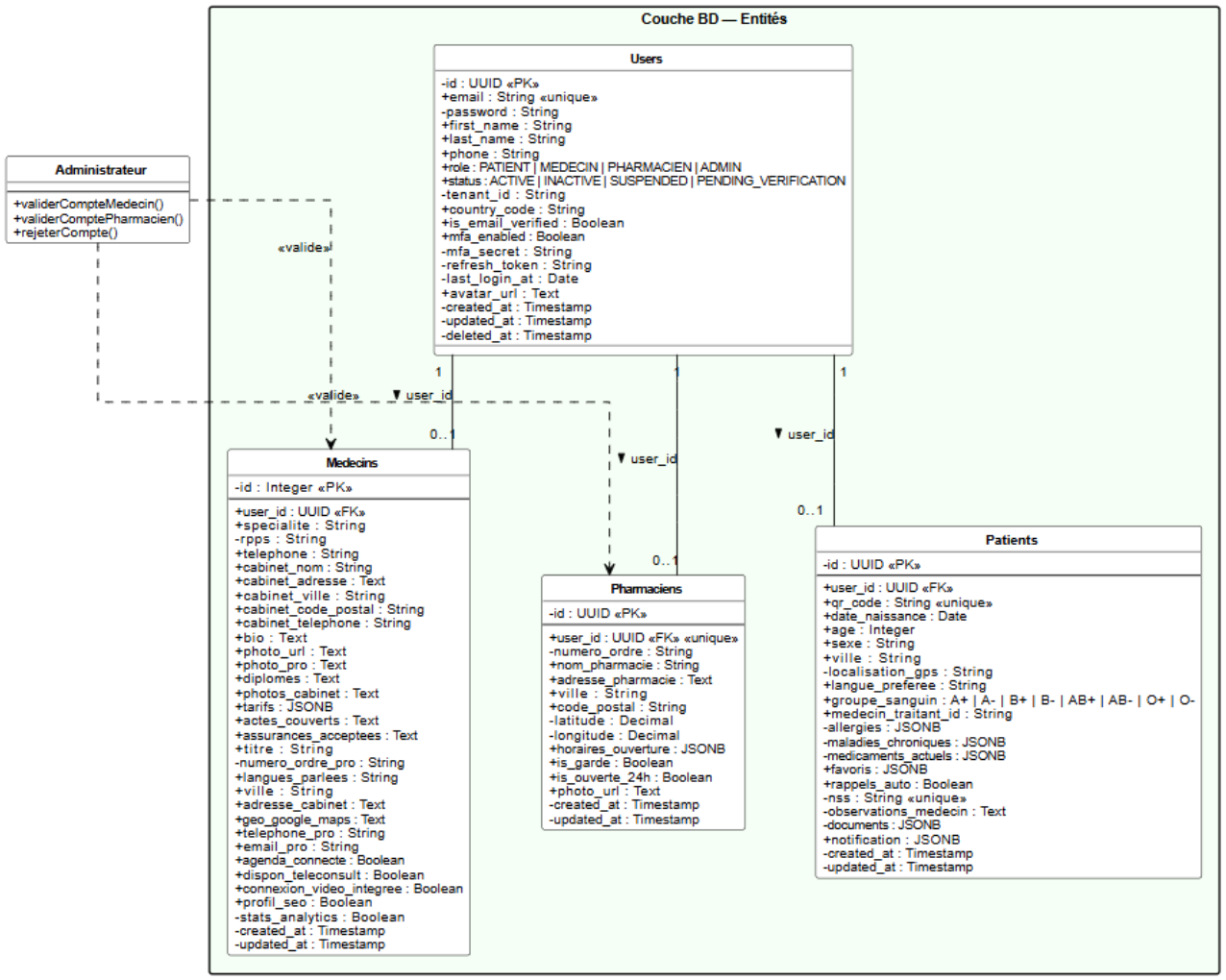


Figure 3.2 –Diagramme de classe Analytique du cas « S'inscrire »

Dans cette figure 3.2 on trouve le diagramme de séquence associé au cas "S'inscrire" :

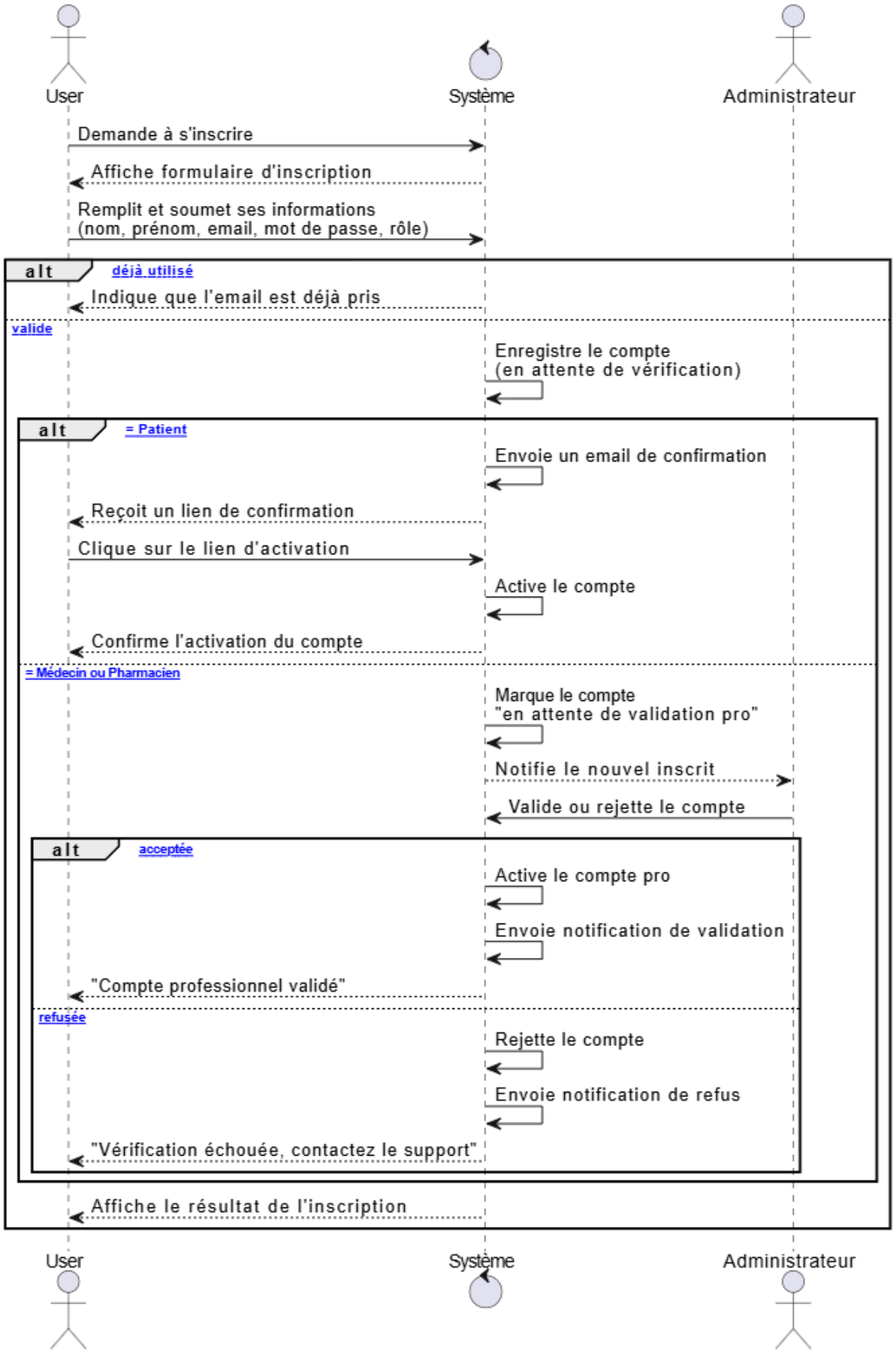


Figure 3.4 –Diagramme de séquence Analytique du cas « S'inscrire »

Le tableau 3.2 illustre en détail le processus de décomposition du cas "S'inscrire" :

Cas	S'inscrire
Acteur	-Patient -Médecin -Pharmacien
Condition préalable	La plateforme est opérationnelle. Adresse mail et mot de passe valides.
Condition résultante	Inscription réussie.
Description de scénario	- La plateforme affiche une page d'inscription. - L'utilisateur remplit ses informations personnelles. - L'utilisateur appuie sur le bouton « S'inscrire ». - La plateforme inspecte les informations. - La plateforme indique un avis de succès.
Scénarii	- La plateforme indique une imprécision si les informations sont inexactes.

Tableau 3.5 – Décomposition du cas « S'inscrire »

Raffinement du cas "S'authentifier"

La figure 3.2 démontre une décomposition de cas d'utilisation "S'authentifier" :

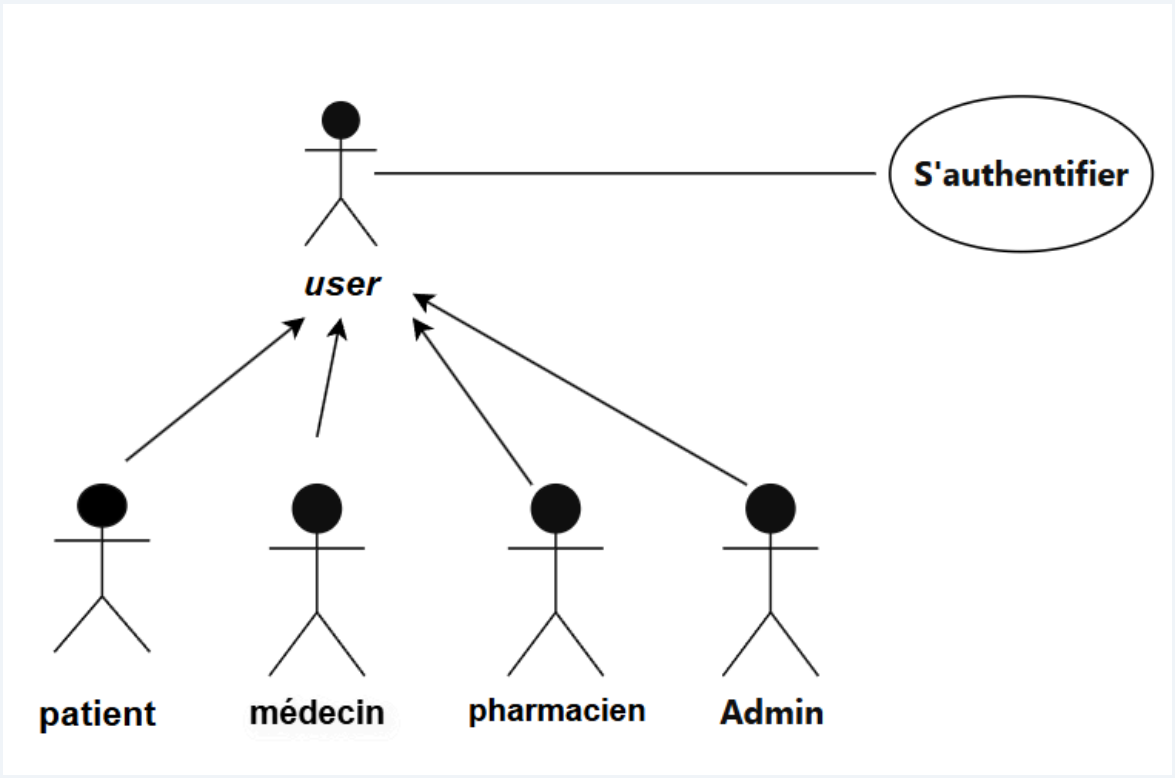


Figure 3.6 – Raffinement du cas d'utilisation « S'authentifier »

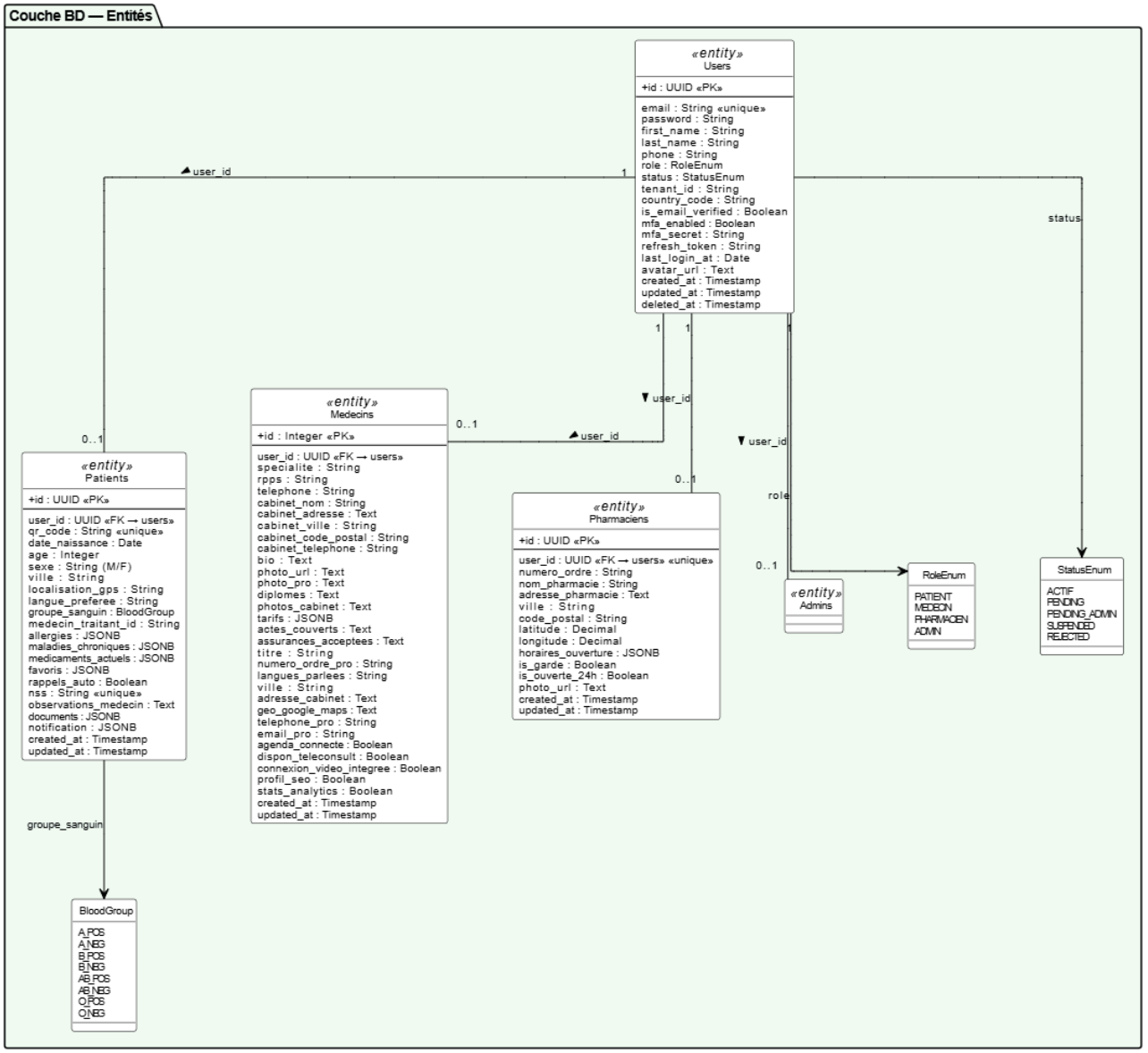


Figure 3.7 –Diagramme de classe Analytique du cas « S'authentifier »

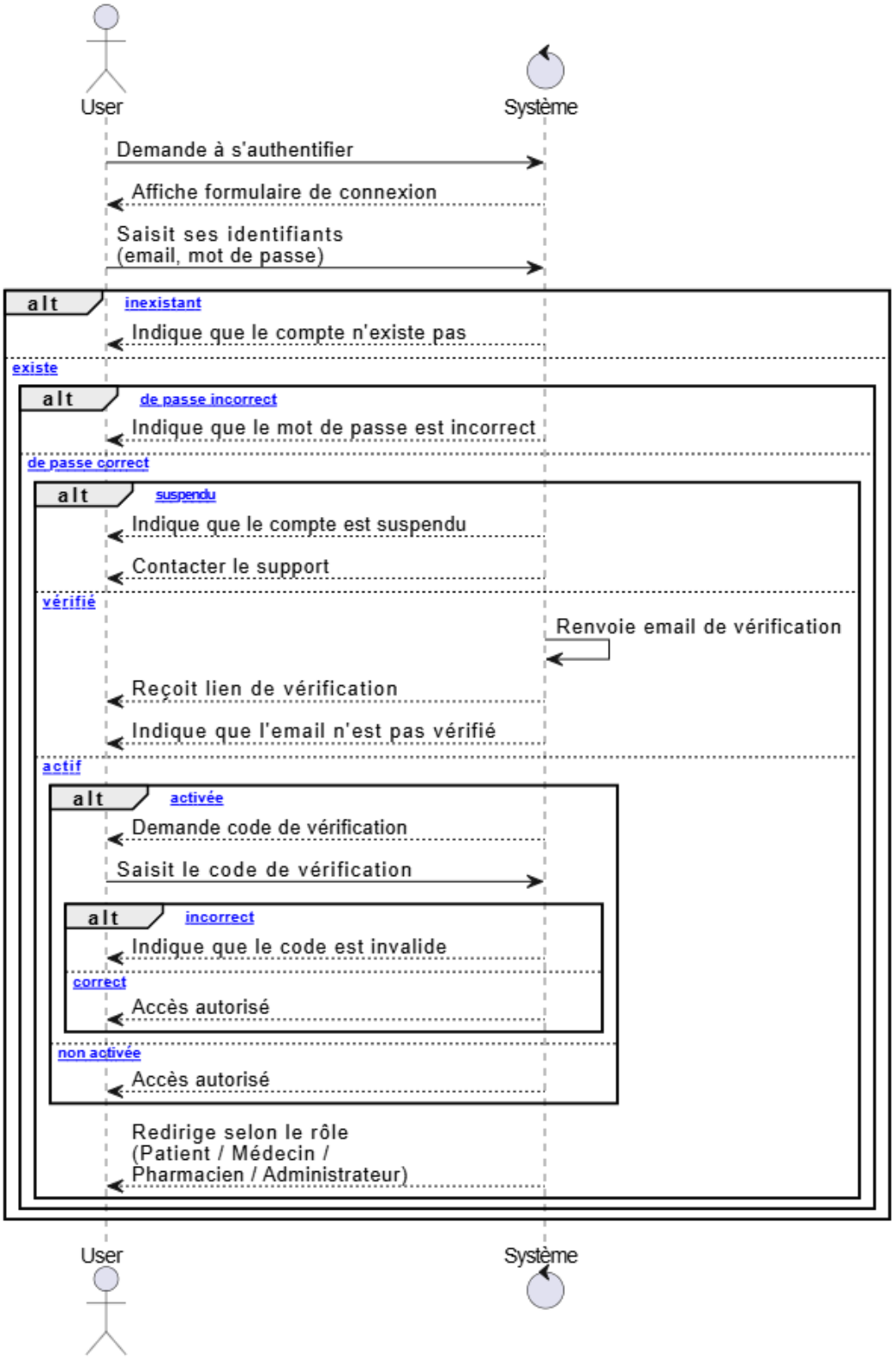


Figure 3.8 –Diagramme de séquence Analytique du cas « S'authentifier »

Le Tableau 3.3 illustre la décomposition détaillée du cas d'utilisation "S'authentifier" :

Cas	S'authentifier
Acteur	-Administrateur -Médecin

	-Patient -Pharmacien
Condition préalable	La plateforme est opérationnelle.
Condition résultante	Utilisateur authentifié.
Description de scénario	- La plateforme affiche la page d'authentification. - L'utilisateur saisit son adresse e-mail et son mot de passe. - La plateforme inspecte l'adresse e-mail et le mot de passe. - La plateforme oriente vers la page suivante.
Scénarii	- La plateforme indique une imprécision si les informations sont inexactes.

Tableau 3.9 – Raffinement du cas « S'authentifier »

3.3.2 Raffinement de Sprint 2

Raffinement du cas "Prendre un rendez-vous"

La figure 3.3 présente la décomposition détaillée du cas "Prendre un rendez-vous" :

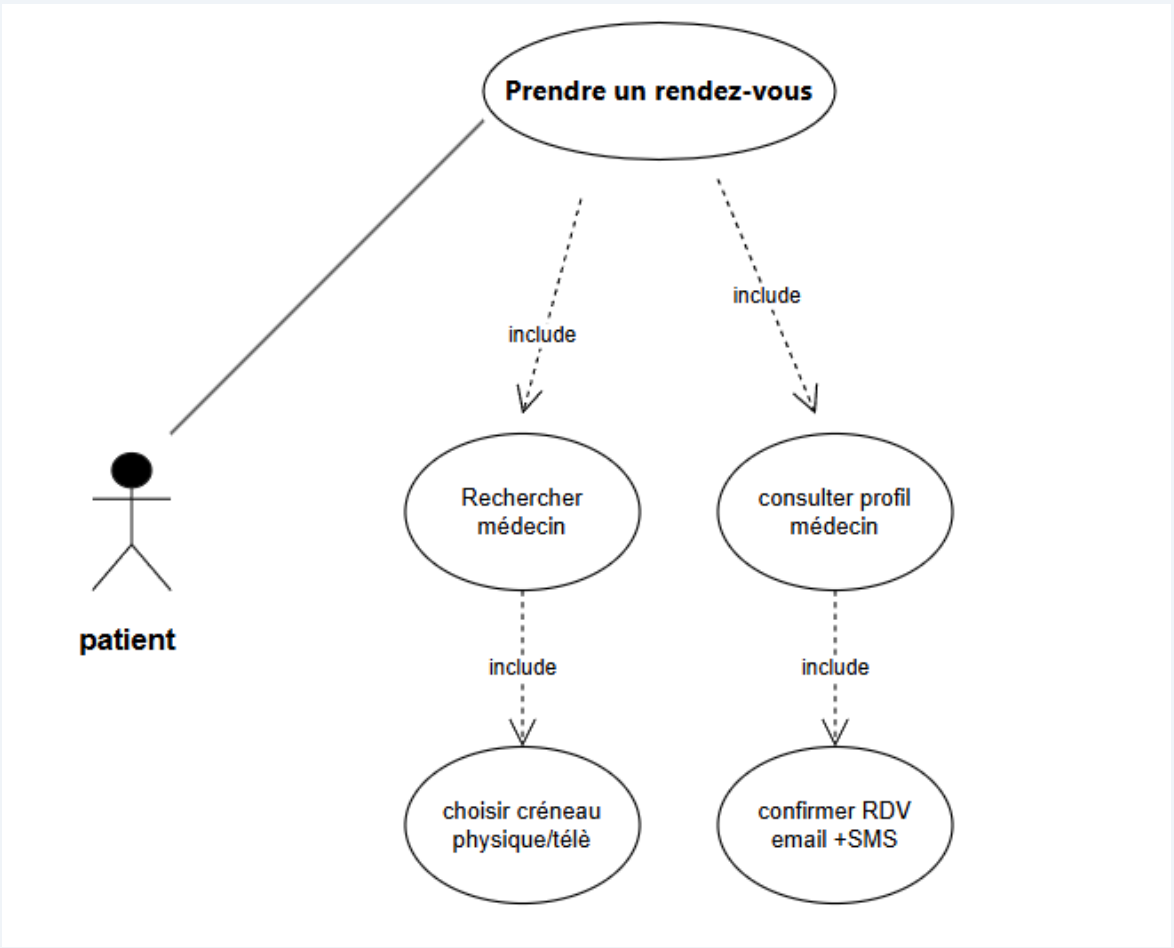


Figure 3.10 – Décomposition de cas d'utilisation « Prendre un rendez-vous »

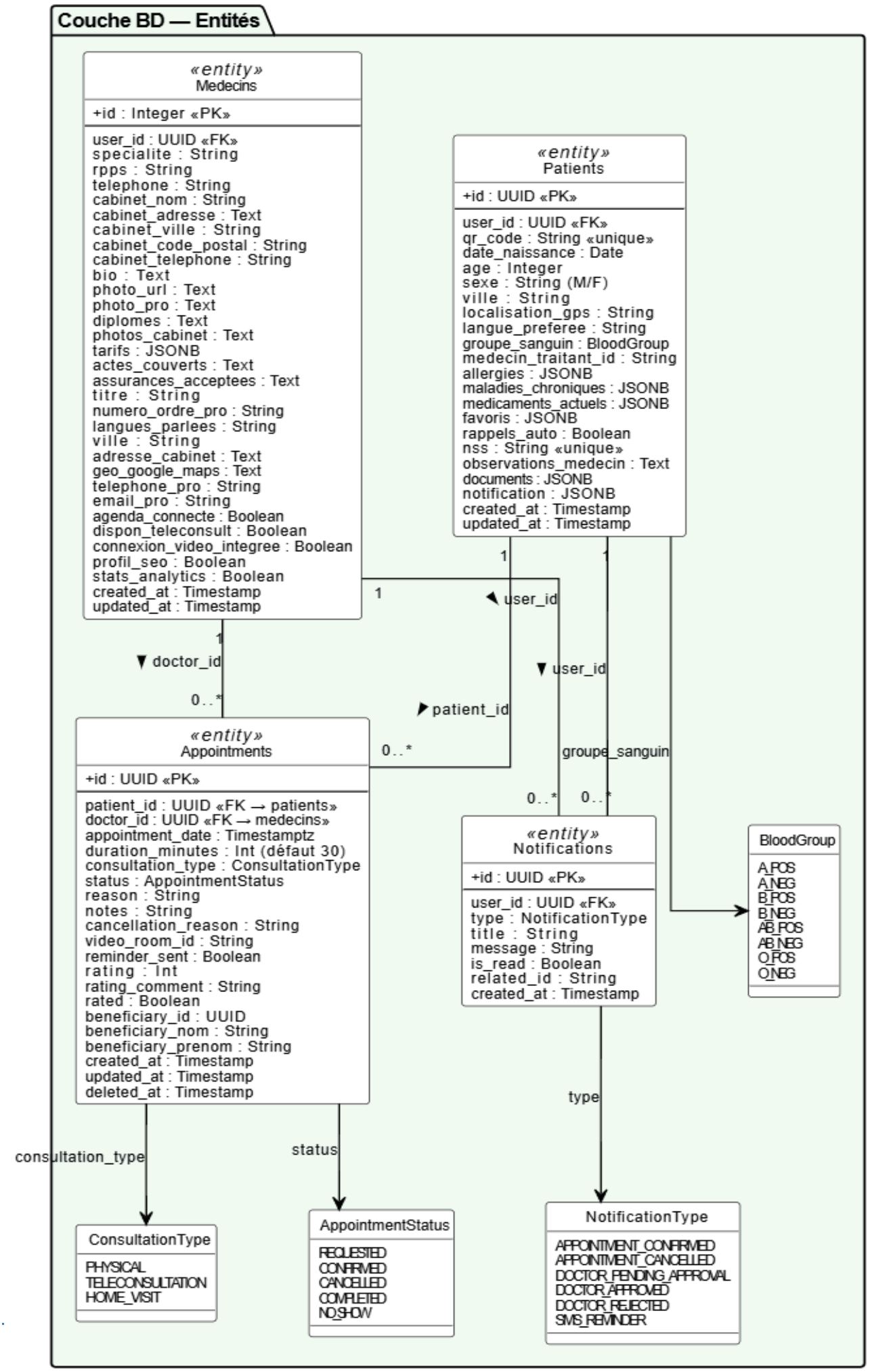
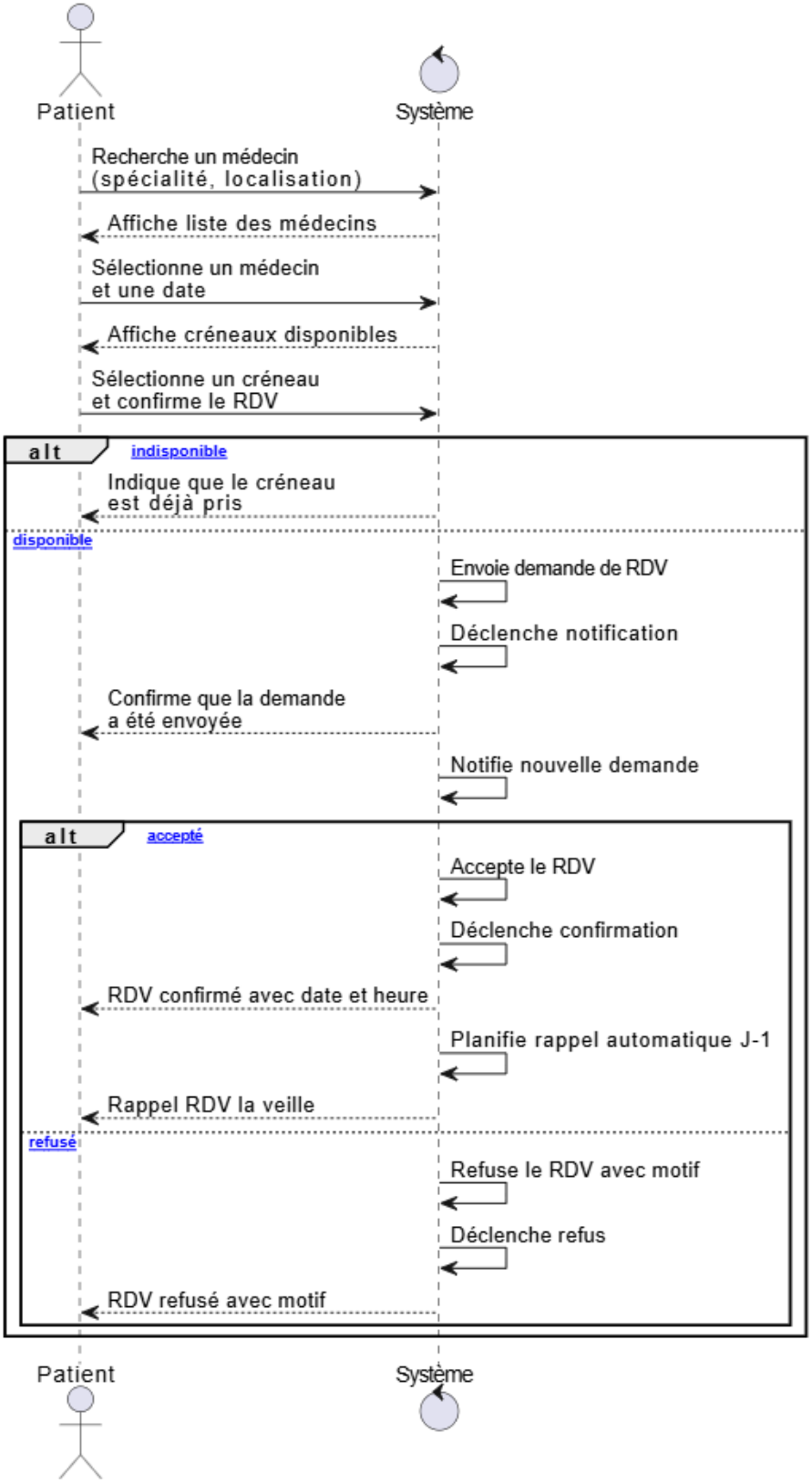

Couche BD — Entités
«entity»
Medecins
+id : Integer «PK»
user_id : UUID «FK»
specialite : String
rpps : String
telephone : String
cabinet_nom : String
cabinet_adresse : Text
cabinet_ville : String
cabinet_code_postal : String
cabinet_telephone : String
bio : Text
photo_url : Text
photo_pro : Text
diplomes : Text
photos_cabinet : Text
tarifs : JSONB
actes_couverts : Text
assurances_acceptees : Text
titre : String
numero_ordre_pro : String
langues_parlees : String
ville : String
adresse_cabinet : Text
geo_google_maps : Text
telephone_pro : String
email_pro : String
agenda_connecte : Boolean
dispon_teleconsult : Boolean
connexion_video_integree : Boolean
profil_seo : Boolean
stats_analytics : Boolean
created_at : Timestamp
updated_at : Timestamp
«entity»
Patients
+id : UUID «PK»
user_id : UUID «FK»
qr_code : String «unique»
date_naissance : Date
age : Integer
sexe : String (M/F)
ville : String
localisation_gps : String
langue_preferee : String
groupe_sanguin : BloodGroup
medecin_traitant_id : String
allergies : JSONB
maladies_chroniques : JSONB
medicaments_actuels : JSONB
favoris : JSONB
rappels_auto : Boolean
nss : String «unique»
observations_medecin : Text
documents : JSONB
notification : JSONB
created_at : Timestamp
updated_at : Timestamp
1
1
1
user_id
1
doctor_id
user_id
0..*
patient_id
«entity»
Appointments
+id : UUID «PK»
patient_id : UUID «FK → patients»
doctor_id : UUID «FK → medecins»
appointment_date : Timestamptz
duration_minutes : Int (défaut 30)
consultation_type : ConsultationType
status : AppointmentStatus
reason : String
notes : String
cancellation_reason : String
video_room_id : String
reminder_sent : Boolean
rating : Int
rating_comment : String
rated : Boolean
beneficiary_id : UUID
beneficiary_nom : String
beneficiary_prenom : String
created_at : Timestamp
updated_at : Timestamp
deleted_at : Timestamp
0..*
groupe_sanguin
0..*
0..*
«entity»
Notifications
+id : UUID «PK»
user_id : UUID «FK»
type : NotificationType
title : String
message : String
is_read : Boolean
related_id : String
created_at : Timestamp
BloodGroup
A_POS
A_NEG
B_POS
B_NEG
AB_POS
AB_NEG
O_POS
O_NEG
type
consultation_type
status
ConsultationType
PHYSICAL
TELECONSULTATION
HOME_VISIT
AppointmentStatus
REQUESTED
CONFIRMED
CANCELLED
COMPLETED
NO_SHOW
NotificationType
APPOINTMENT_CONFIRMED
APPOINTMENT_CANCELLED
DOCTOR_PENDING_APPROVAL
DOCTOR_APPROVED
DOCTOR_REJECTED
SMS_REMINDER

Figure 3.11–Diagramme de classe Analytique du cas « Prendre un rendez-vous »



3.12-Diagramme de séquence Analytique du cas « Prendre un rendez-vous »

Le Tableau 3.4 présente la décomposition détaillée du cas "Prendre un rendez-vous" :

Cas	Prendre un rendez-vous
Acteur	-Patient
Condition préalable	L'utilisateur est authentifié et la plateforme est opérationnelle.
Condition résultante	Rendez-vous créé et confirmé.
Description de scénario	- Le patient recherche un médecin par spécialité, localisation et disponibilité. - Le patient consulte le profil du médecin (photo, avis, tarifs). - Le patient sélectionne un créneau disponible en temps réel (physique ou téléconsultation). - Le patient confirme la prise de rendez-vous. - La plateforme envoie une confirmation par email et SMS. - Un rappel automatique est envoyé 24h avant le RDV avec le lien de téléconsultation.
Scénarii	- Aucun créneau disponible : la plateforme affiche un message d'indisponibilité.

Tableau 3.4 – Raffinement du cas « Prendre un rendez-vous »

Raffinement du cas "Gérer les rendez-vous"

La figure 3.4 présente le raffinement du cas d'utilisation "Gérer les rendez-vous" :

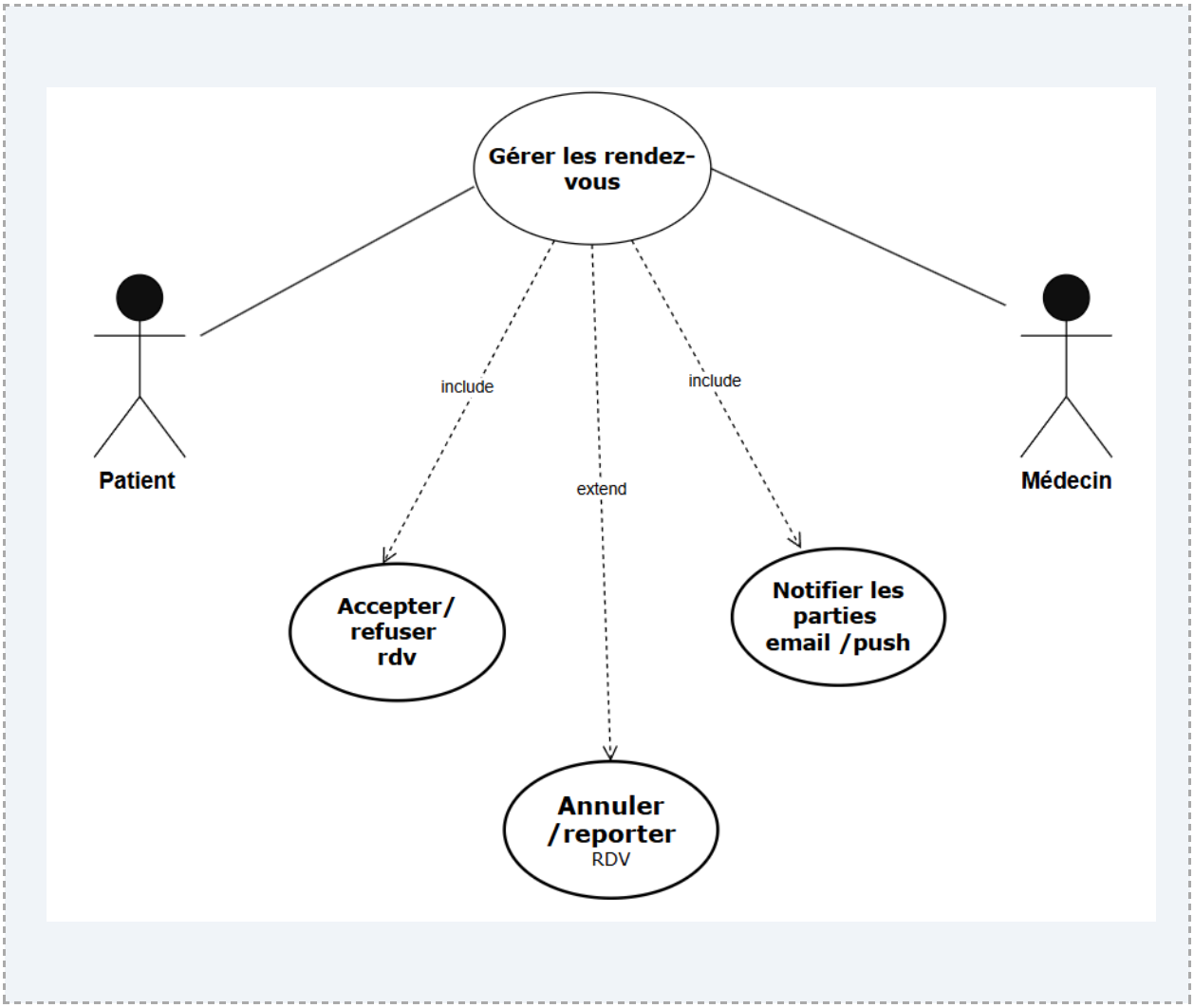


Figure 3.13 – Raffinement du cas « Gérer les rendez-vous »

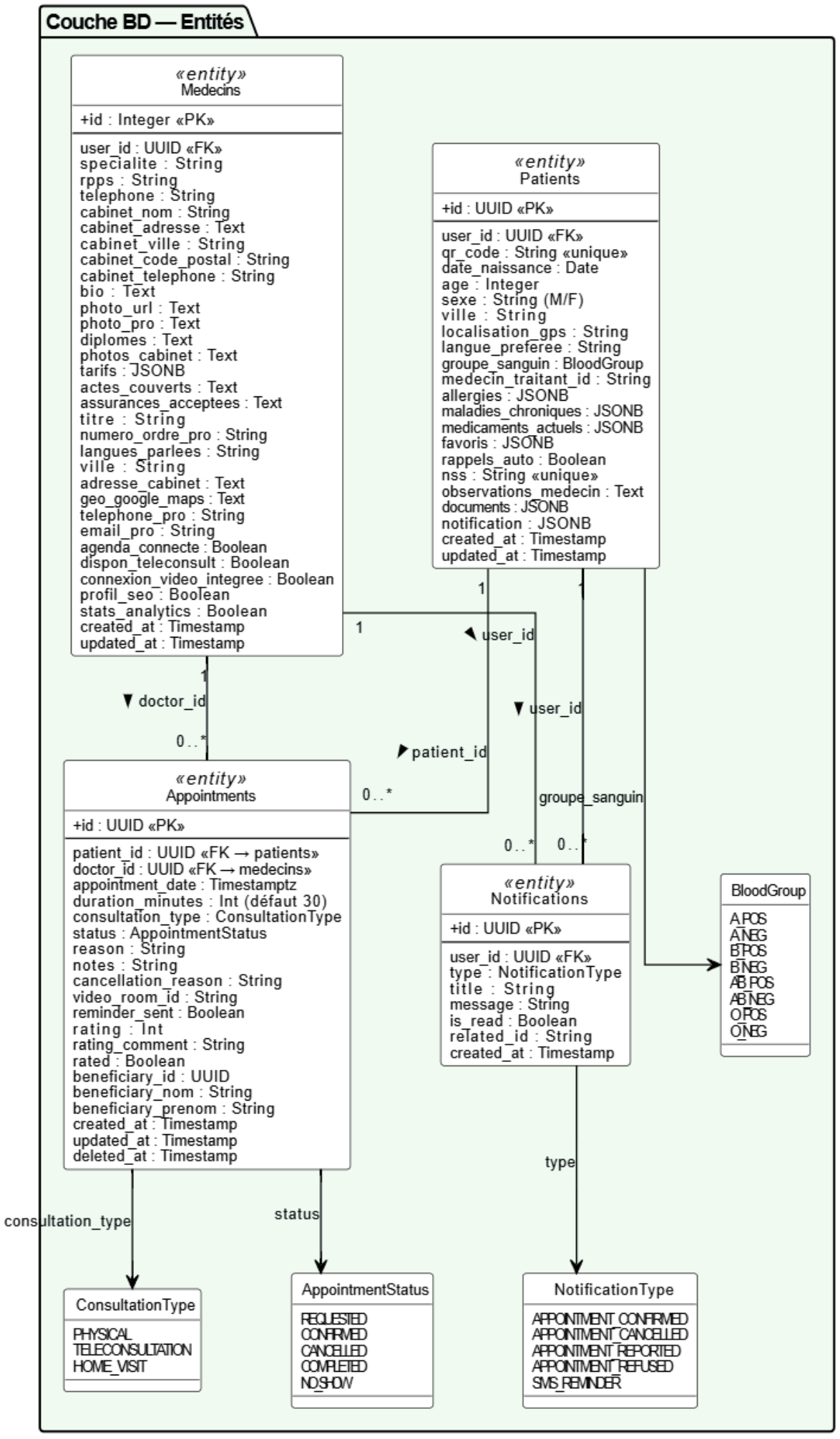

Couche BD — Entités
«entity»
Medecins
+id : Integer «PK»
user_id : UUID «FK»
specialite : String
rpps : String
telephone : String
cabinet_nom : String
cabinet_adresse : Text
cabinet_ville : String
cabinet_code_postal : String
cabinet_telephone : String
bio : Text
photo_url : Text
photo_pro : Text
diplomes : Text
photos_cabinet : Text
tarifs : JSONB
actes_couverts : Text
assurances_acceptees : Text
titre : String
numero_ordre_pro : String
langues_parlees : String
ville : String
adresse_cabinet : Text
geo_google_maps : Text
telephone_pro : String
email_pro : String
agenda_connecte : Boolean
dispon_teleconsult : Boolean
connexion_video_integree : Boolean
profil_seo : Boolean
stats_analytics : Boolean
created_at : Timestamp
updated_at : Timestamp
«entity»
Patients
+id : UUID «PK»
user_id : UUID «FK»
qr_code : String «unique»
date_naissance : Date
age : Integer
sexe : String (M/F)
ville : String
localisation_gps : String
langue_preferee : String
groupe_sanguin : BloodGroup
medecin_traitant_id : String
allergies : JSONB
maladies_chroniques : JSONB
medicaments_actuels : JSONB
favoris : JSONB
rappels_auto : Boolean
nss : String «unique»
observations_medecin : Text
documents : JSONB
notification : JSONB
created_at : Timestamp
updated_at : Timestamp
1
1
user_id
1
doctor_id
0..*
user_id
patient_id
0..*
groupe_sanguin
«entity»
Appointments
+id : UUID «PK»
patient_id : UUID «FK → patients»
doctor_id : UUID «FK → medecins»
appointment_date : Timestamptz
duration_minutes : Int (défaut 30)
consultation_type : ConsultationType
status : AppointmentStatus
reason : String
notes : String
cancellation_reason : String
video_room_id : String
reminder_sent : Boolean
rating : Int
rating_comment : String
rated : Boolean
beneficiary_id : UUID
beneficiary_nom : String
beneficiary_prenom : String
created_at : Timestamp
updated_at : Timestamp
deleted_at : Timestamp
0..*
0..*
«entity»
Notifications
+id : UUID «PK»
user_id : UUID «FK»
type : NotificationType
title : String
message : String
is_read : Boolean
related_id : String
created_at : Timestamp
BloodGroup
A_POS
A_NEG
B_POS
B_NEG
AB_POS
AB_NEG
O_POS
O_NEG
type
consultation_type
status
ConsultationType
PHYSICAL
TELECONSULTATION
HOME_VISIT
AppointmentStatus
REQUESTED
CONFIRMED
CANCELLED
COMPLETED
NO_SHOW
NotificationType
APPOINTMENT_CONFIRMED
APPOINTMENT_CANCELLED
APPOINTMENT_REPORTED
APPOINTMENT_REFUSED
SMS_REMINDER

Figure 3.14 –Diagramme de classe Analytique **du cas "Gérer les rendez-vous"**

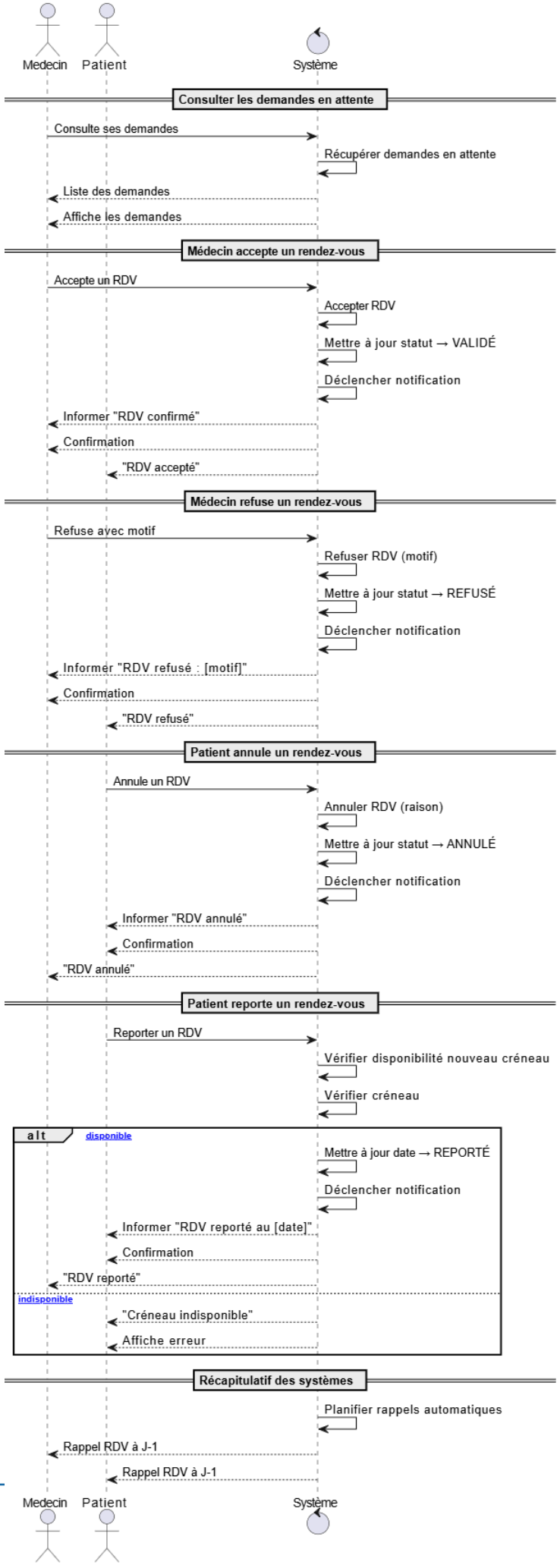

Medecin
Patient
Système
Consulter les demandes en attente
Consulte ses demandes
Récupérer demandes en attente
Liste des demandes
Affiche les demandes
Médecin accepte un rendez-vous
Accepte un RDV
Accepter RDV
Mettre à jour statut → VALIDÉ
Déclencher notification
Informer "RDV confirmé"
Confirmation
"RDV accepté"
Médecin refuse un rendez-vous
Refuse avec motif
Refuser RDV (motif)
Mettre à jour statut → REFUSÉ
Déclencher notification
Informer "RDV refusé : [motif]"
Confirmation
"RDV refusé"
Patient annule un rendez-vous
Annule un RDV
Annuler RDV (raison)
Mettre à jour statut → ANNULÉ
Déclencher notification
Informer "RDV annulé"
Confirmation
"RDV annulé"
Patient reporte un rendez-vous
Reporter un RDV
Vérifier disponibilité nouveau créneau
Vérifier créneau
alt
disponible
Mettre à jour date → REPORTÉ
Déclencher notification
Informer "RDV reporté au [date]"
Confirmation
"RDV reporté"
indisponible
"Créneau indisponible"
Affiche erreur
Récapitulatif des systèmes
Planifier rappels automatiques
Rappel RDV à J-1
Rappel RDV à J-1
Medecin
Patient
Système

***Diagramme de séquence Analytique* du cas "Gérer les rendez-vous"**

Le Tableau 3.5 présente le raffinement du cas "Gérer les rendez-vous" :

Cas	Gérer les rendez-vous
Acteur	-Patient -Médecin
Condition préalable	L'utilisateur est authentifié.
Condition résultante	Rendez-vous modifié ou annulé avec succès.
Description de scénario	- Le médecin reçoit une notification instantanée lors d'une demande de RDV. - Le médecin accepte ou refuse la demande en 1 clic (statut VALIDÉ / REFUSÉ). - Le patient consulte ses rendez-vous depuis son espace personnel. - Le patient peut annuler ou reporter un RDV depuis son espace. - La plateforme notifie les deux parties de tout changement.
Scénarii	- Annulation tardive : la plateforme affiche un avertissement. - Refus médecin : notification envoyée au patient.

Tableau 3.5 – Raffinement du cas « Gérer les rendez-vous »

3.4 Conception de Release 1

3.4.1 Conception de Sprint 0

Conception du cas "S'inscrire"

Figure 3.15 – Diagramme de classe conceptuel du cas « S'inscrire »

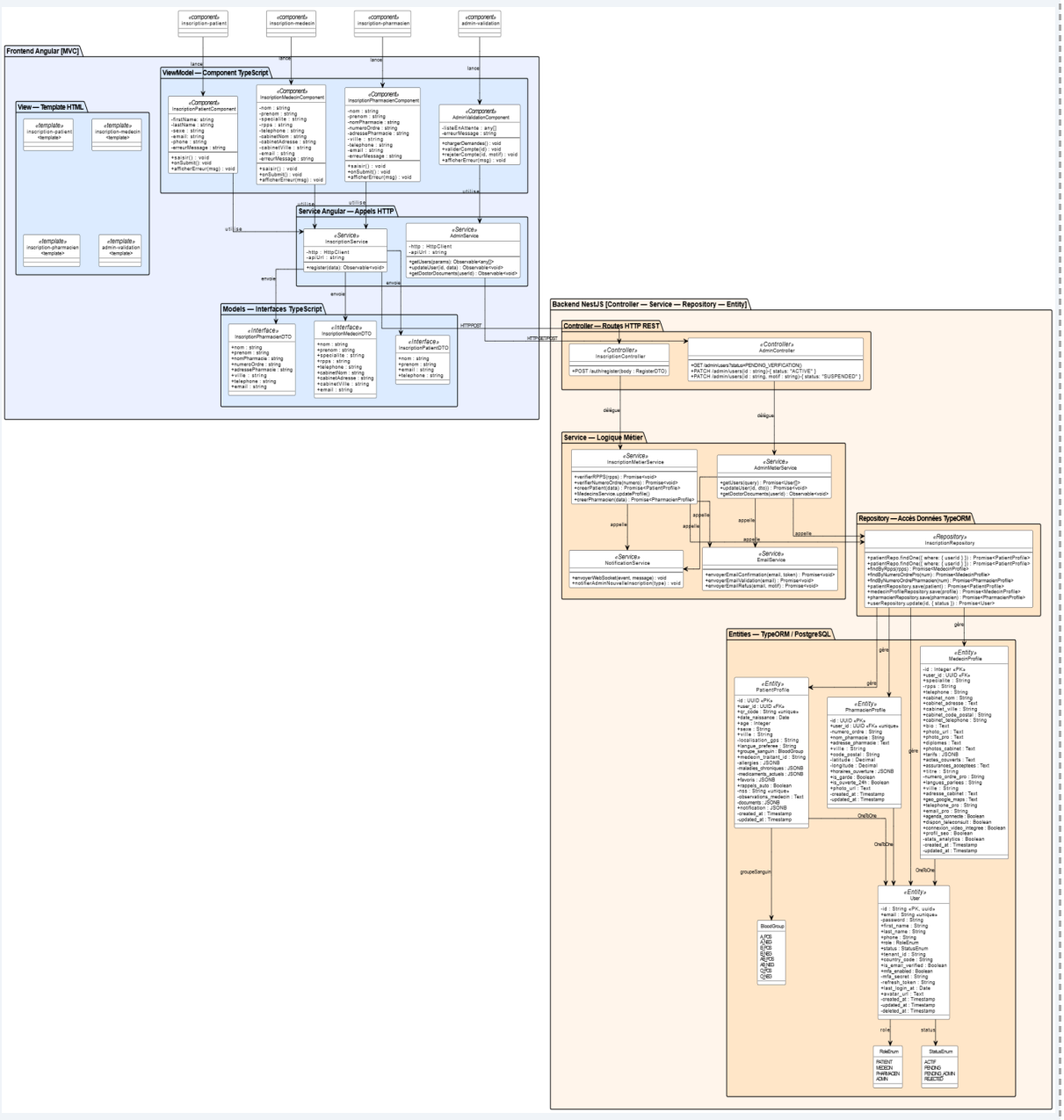

«component» inscription-patient
«component» inscription-medecin
«component» inscription-pharmacien
«component» admin-validation
lance
Frontend Angular [MVC]
ViewModel — Component TypeScript
«Component» InscriptionPatientComponent
-firstName: string
-lastName: string
-sexe : string
-email : string
-phone : string
-erreurMessage : string
+saisir() : void
+onSubmit() : void
+afficherErreur(msg) : void
«Component» InscriptionMedecinComponent
-nom : string
-prenom : string
-specialite : string
-rpps : string
-telephone : string
-adresseMedecin : string
-cabinetAdresse : string
-cabinetVille : string
-email : string
-erreurMessage : string
+saisir() : void
+onSubmit() : void
+afficherErreur(msg) : void
«Component» InscriptionPharmacienComponent
-nom : string
-prenom : string
-nomPharmacie : string
-numeroOrdre : string
-adressePharmacie : string
-ville : string
-telephone : string
-email : string
-erreurMessage : string
+saisir() : void
+onSubmit() : void
+afficherErreur(msg) : void
«Component» AdminValidationComponent
-listeEnAttente : any[]
-erreurMessage : string
+chargerDemandes() : void
+validerCompte(id) : void
+refuserCompte(id, motif) : void
+afficherErreur(msg) : void
View — Template HTML
«template» inscription-patient <template>
«template» inscription-medecin <template>
«template» inscription-pharmacien <template>
«template» admin-validation <template>
utilise
Service Angular — Appels HTTP
«Service» InscriptionService
-http : HttpClient
-apiUrl : string
+register(data): Observable<void>
«Service» AdminService
-http : HttpClient
-apiUrl : string
+getUsers(params): Observable<any[]>
+updateUser(id, data): Observable<void>
+getDoctorDocuments(userId) : Observable<void>
envoie
Models — Interfaces TypeScript
«Interface» InscriptionPharmacienDTO
+nom : string
+prenom : string
+nomPharmacie : string
+numeroOrdre : string
+adressePharmacie : string
+ville : string
+telephone : string
+email : string
«Interface» InscriptionMedecinDTO
+nom : string
+prenom : string
+specialite : string
+rpps : string
+telephone : string
+cabinetNom : string
+cabinetAdresse : string
+cabinetVille : string
+email : string
«Interface» InscriptionPatientDTO
+nom : string
+prenom : string
+email : string
+telephone : string
+HTTP/POST
Backend NestJS [Controller — Service — Repository — Entity]
Controller — Routes HTTP REST
«Controller» InscriptionController
+POST /auth/register(body : RegisterDTO)
«Controller» AdminController
+GET /admin/users?status=PENDING_VERIFICATION()
+PATCH /admin/users/:id : string-{ status: "ACTIVE" }
+PATCH /admin/users/:id : string, motif : string)-{ status: "SUSPENDED" }
délègue
Service — Logique Métier
«Service» InscriptionMetierService
+verifierRPPS(rpps) : Promise<void>
+verifierNumeroOrdre(numero) : Promise<void>
+creerPatient(data) : Promise<PatientProfile>
+fileDeclenchService.updateProfile()
+creerPharmacien(data) : Promise<PharmacienProfile>
«Service» AdminMetierService
+getUsers(query) : Promise<User[]>
+updateUser(id, dto)) : Promise<void>
+getDoctorDocuments(userId) : Observable<void>
«Service» NotificationService
+envoyerWebSocket(event, message) : void
+notifierAdminNouvelleInscription(type) : void
«Service» EmailService
+envoyerEmailConfirmation(email, token) : Promise<void>
+envoyerEmailValidation(email) : Promise<void>
+envoyerEmailRefus(email, motif) : Promise<void>
appelle
Repository — Accès Données TypeORM
«Repository» InscriptionRepository
+patientRepo.findOne({ where: { userId } }) : Promise<PatientProfile>
+patientRepo.findOne({ where: { userId } }) : Promise<PatientProfile>
+findByRpps(rpps) : Promise<MedecinProfile>
+findByNumeroOrdrePro(num) : Promise<MedecinProfile>
+findByNumeroOrdrePharmacien(num) : Promise<PharmacienProfile>
+patientRepository.save(patient) : Promise<PatientProfile>
+medecinRepository.save(medecin) : Promise<MedecinProfile>
+pharmacienRepository.save(pharmacien) : Promise<PharmacienProfile>
+userRepository.update(id, { status }) : Promise<User>
gère
Entities — TypeORM / PostgreSQL
«Entity» PatientProfile
-id : UUID «PK»
+user_id : UUID «FK»
+qr_code : String «unique»
+date_naissance : Date
+age : Integer
+sexe : String
+ville : String
-localisation_gps : String
+groupe_sanguin : BloodGroup
+medecin_traitant_id : String
-allergies : JSONB
-maladies_chroniques : JSONB
-medicaments_actuels : JSONB
+favoris : JSONB
+rappels_auto : Boolean
-rss : String «unique»
-observations_medecin : Text
-documents : JSONB
+notification : JSONB
-created_at : Timestamp
-updated_at : Timestamp
«Entity» PharmacienProfile
-id : UUID «PK»
+user_id : UUID «FK» «unique»
-numero_ordre : String
+nom_pharmacie : String
+adresse_pharmacie : Text
+ville : String
+code_postal : String
-latitude : Decimal
-longitude : Decimal
+horaires_ouverture : JSONB
+is_garde : Boolean
+is_ouverte_24h : Boolean
+rss : String «unique»
-created_at : Timestamp
-updated_at : Timestamp
«Entity» MedecinProfile
-id : Integer «PK»
+user_id : UUID «FK»
+specialite : String
-rpps : String
+telephone : String
+cabinet_nom : String
+cabinet_adresse : Text
+cabinet_ville : String
+cabinet_code_postal : String
+cabinet_telephone : String
+bio : Text
+photo_url : Text
+photo_pro : Text
+diplomes : Text
+photos_cabinet : Text
+tarifs : JSONB
+actes_ouverts : Text
+assurances_acceptees : Text
+titre : String
+numero_ordre_pro : String
+langues_parlees : String
+rss : String
+adresse_cabinet : Text
+geo_google_maps : Text
+telephone_pro : String
+email_pro : String
+agenda_connecte : Boolean
+dispon_teleconsult : Boolean
+connexion_video_integree : Boolean
+profil_tao : Boolean
-stats_analytics : Boolean
-created_at : Timestamp
-updated_at : Timestamp
OneToOne
groupeSanguin
BloodGroup
A_POS
A_NEG
B_POS
B_NEG
AB_POS
AB_NEG
O_POS
O_NEG
«Entity» User
-id : String «PK, uuid»
+email : String «unique»
-password : String
+first_name : String
+last_name : String
+phone : String
+role : RoleEnum
+status : StatusEnum
+tenant_id : String
+country_code : String
+is_email_verified : Boolean
+mfa_enabled : Boolean
-mfa_secret : String
-refresh_token : String
+last_login_at : Date
+avatar_url : Text
-created_at : Timestamp
-updated_at : Timestamp
-deleted_at : Timestamp
role
status
RoleEnum
PATIENT
MEDECIN
PHARMACIEN
ADMIN
StatutEnum
ACTIF
PENDING
PENDING_ADMIN
REJECTED



Figure 3.17 – Diagramme de séquence conceptuel du cas « S'inscrire »

Conception du cas "S'authentifier"

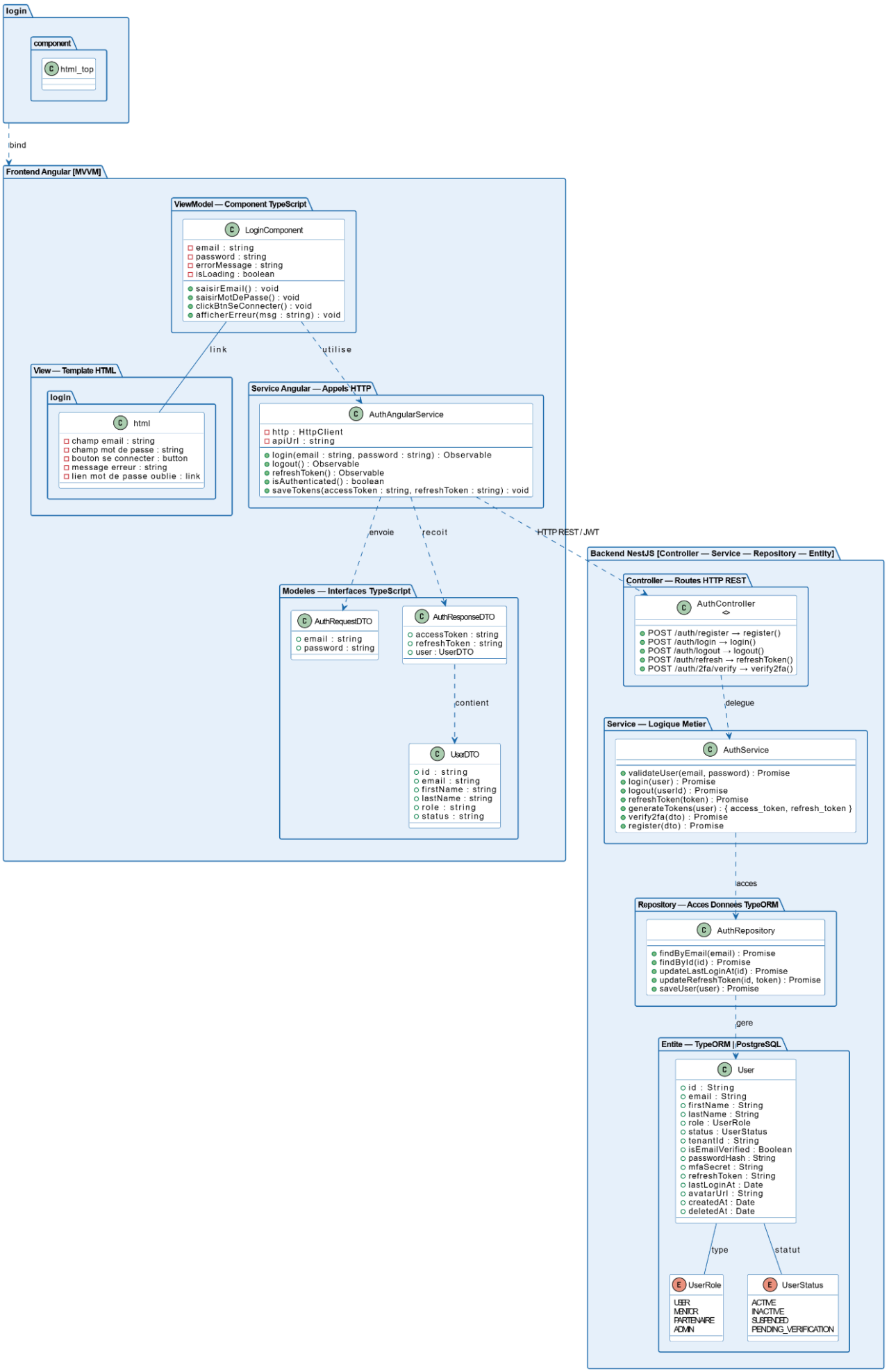

login
component
html_top
bind
Frontend Angular [MVVM]
ViewModel — Component TypeScript
LoginComponent
email : string
password : string
errorMessage : string
isLoading : boolean
saisirEmail() : void
saisirMotDePasse() : void
clickBtnSeConnecter() : void
afficherErreur(msg : string) : void
link
utilise
View — Template HTML
login
html
champ email : string
champ mot de passe : string
bouton se connecter : button
message erreur : string
lien mot de passe oublie : link
Service Angular — Appels HTTP
AuthAngularService
http : HttpClient
apiUrl : string
login(email : string, password : string) : Observable
logout() : Observable
refreshToken() : Observable
isAuthenticated() : boolean
saveTokens(accessToken : string, refreshToken : string) : void
envoie
recoit
HTTP REST / JWT
Modeles — Interfaces TypeScript
AuthRequestDTO
email : string
password : string
AuthResponseDTO
accessToken : string
refreshToken : string
user : UserDTO
contient
UserDTO
id : string
email : string
firstName : string
lastName : string
role : string
status : string
Backend NestJS [Controller — Service — Repository — Entity]
Controller — Routes HTTP REST
AuthController
POST /auth/register → register()
POST /auth/login → login()
POST /auth/logout → logout()
POST /auth/refresh → refreshToken()
POST /auth/2fa/verify → verify2fa()
delegue
Service — Logique Metier
AuthService
validateUser(email, password) : Promise
login(user) : Promise
logout(userId) : Promise
refreshToken(token) : Promise
generateTokens(user) : { access_token, refresh_token }
verify2fa(dto) : Promise
register(dto) : Promise
acces
Repository — Acces Donnees TypeORM
AuthRepository
findByEmail(email) : Promise
findById(id) : Promise
updateLastLoginAt(id) : Promise
updateRefreshToken(id, token) : Promise
saveUser(user) : Promise
gere
Entite — TypeORM | PostgreSQL
User
id : String
email : String
firstName : String
lastName : String
role : UserRole
status : UserStatus
tenantId : String
isEmailVerified : Boolean
passwordHash : String
mfaSecret : String
refreshToken : String
lastLoginAt : Date
avatarUrl : String
createdAt : Date
deletedAt : Date
type
statut
UserRole
USER
MENTOR
PARTENAIRE
ADMIN
UserStatus
ACTIVE
INACTIVE
SUSPENDED
PENDING_VERIFICATION

Figure 3.18 – Diagramme de classe Conceptuel du cas « S'authentifier »

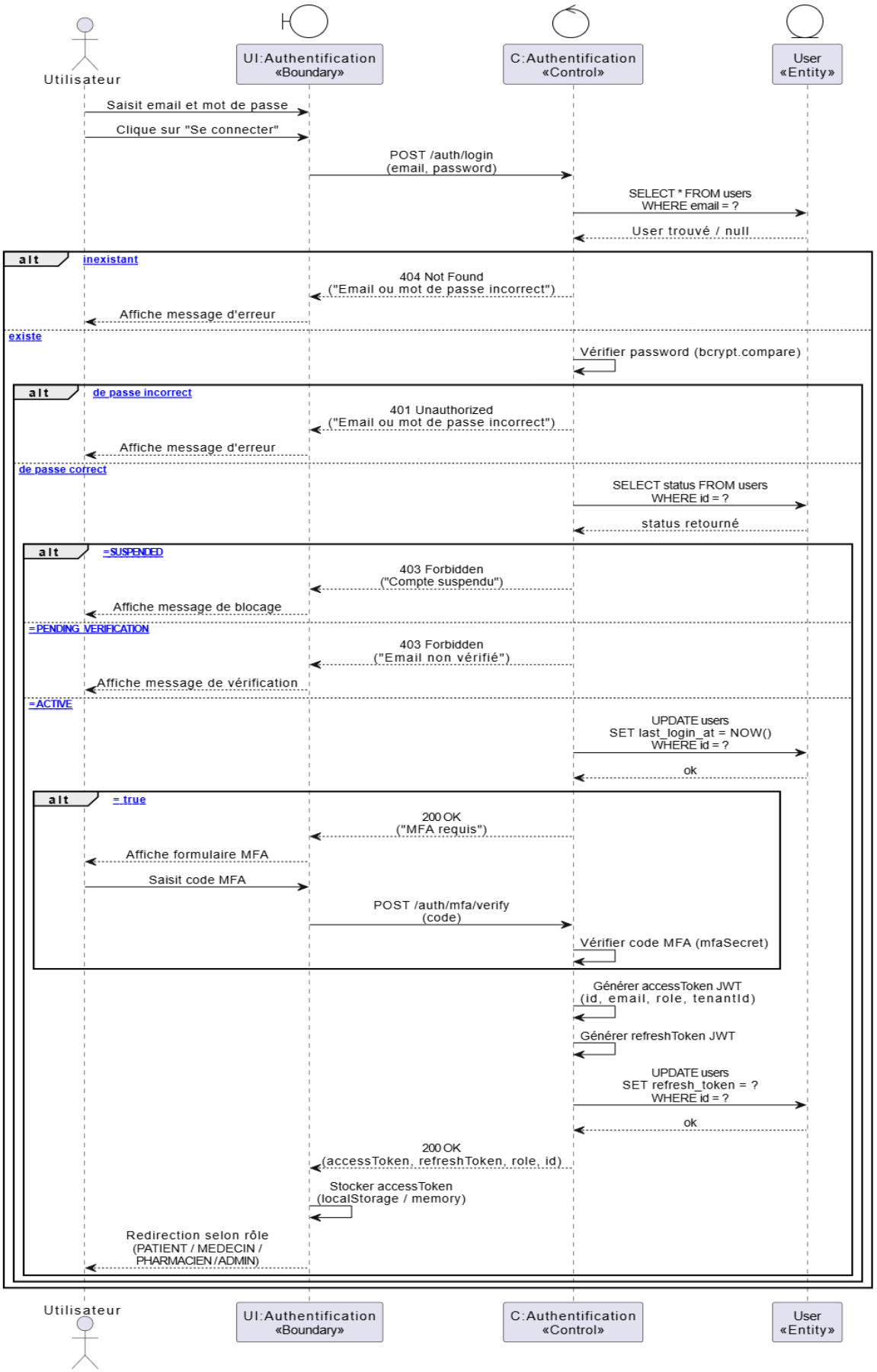


Figure 3.19 – Diagramme de séquence Conceptuel du cas « S'authentifier »

3.4.2 Conception de Sprint 1

Conception du cas "Prendre un rendez-vous"

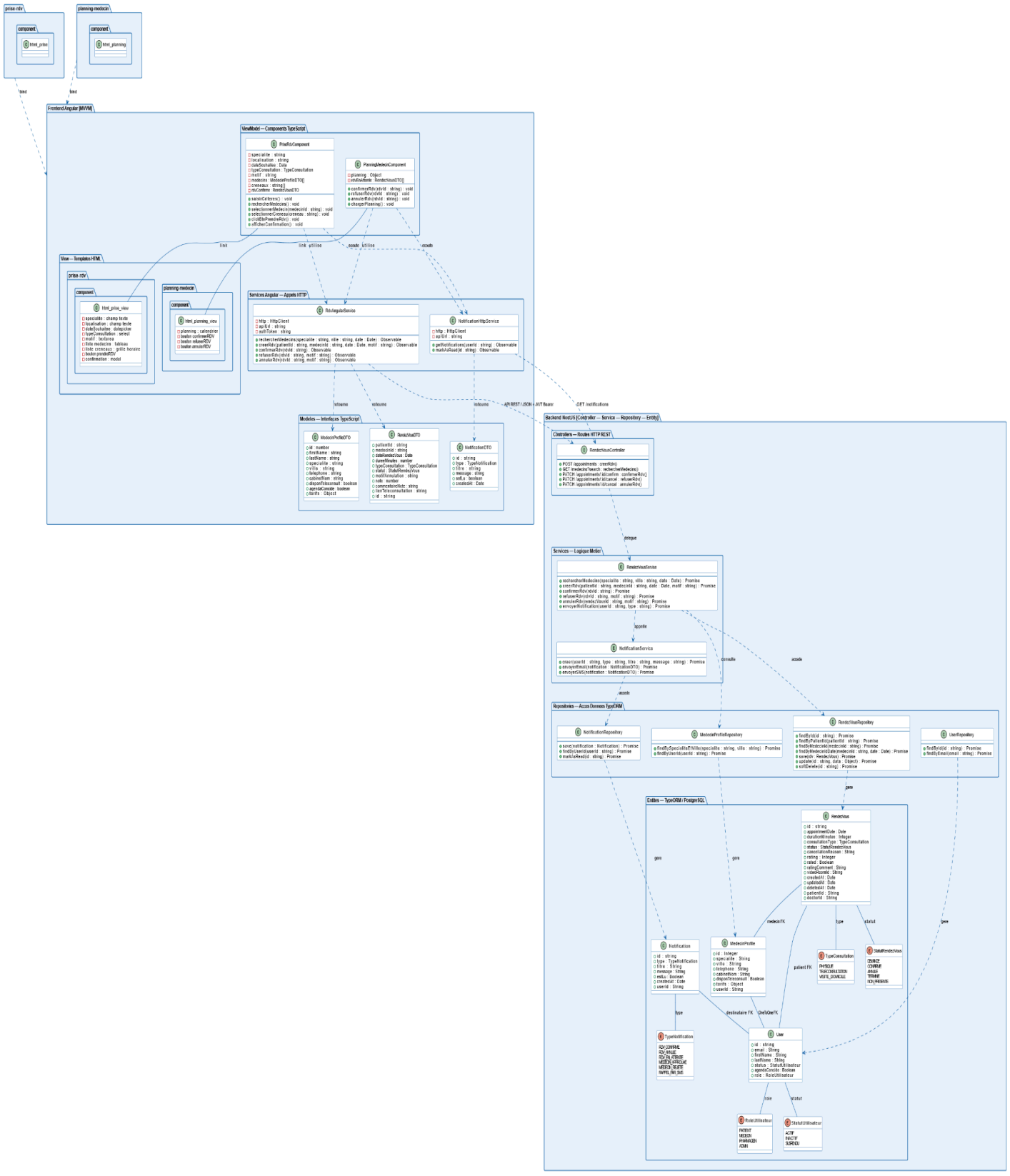


Figure 3.20 – Diagramme de classe Conceptuel du cas « Prendre un rendez-vous »

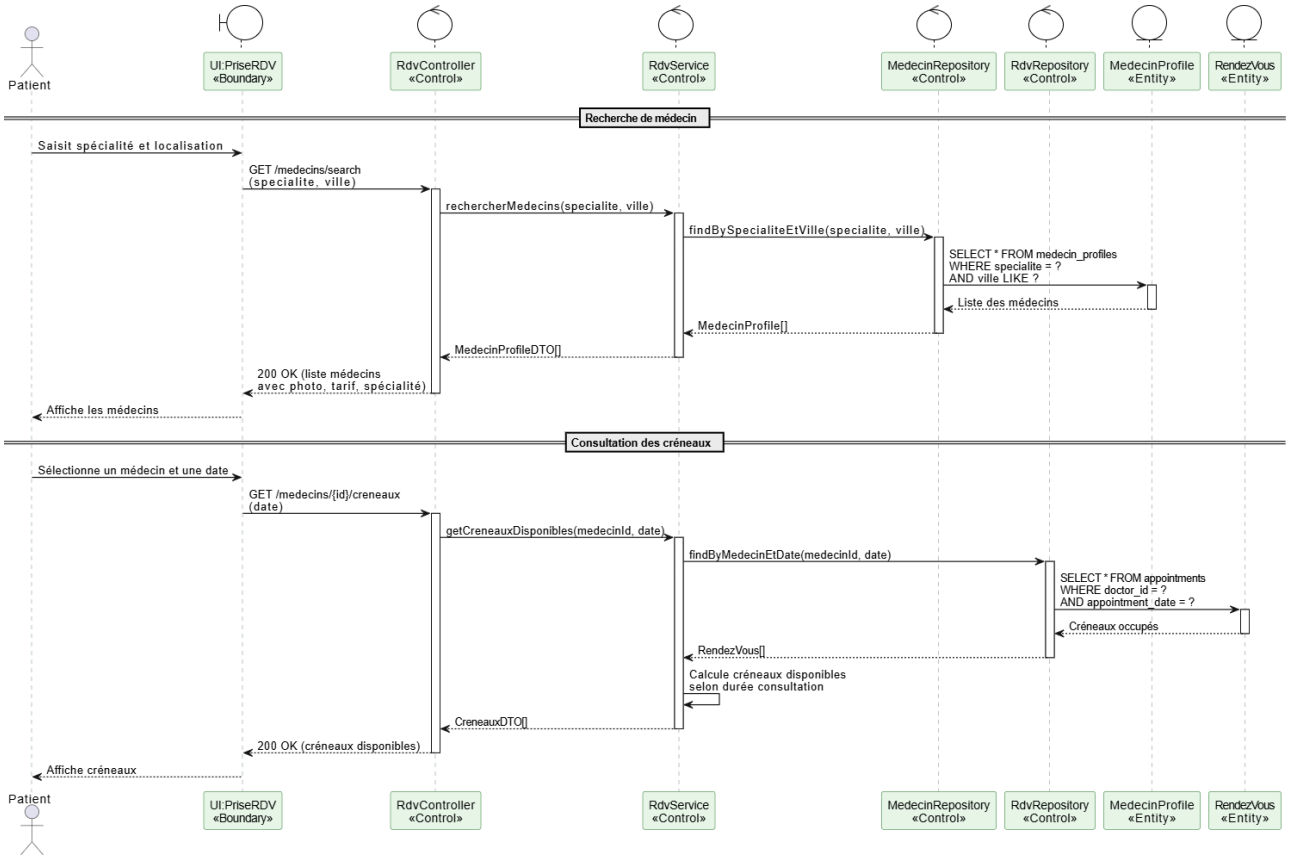


Diagramme de Séquence conceptuel — Prendre un RDV (Partie 1)\n[Recherche médecin + Consultation créneaux]

Diagramme de Séquence conceptuel — Prendre un RDV (Partie 2)\n[Prise RDV + Médecin accepte ou refuse]

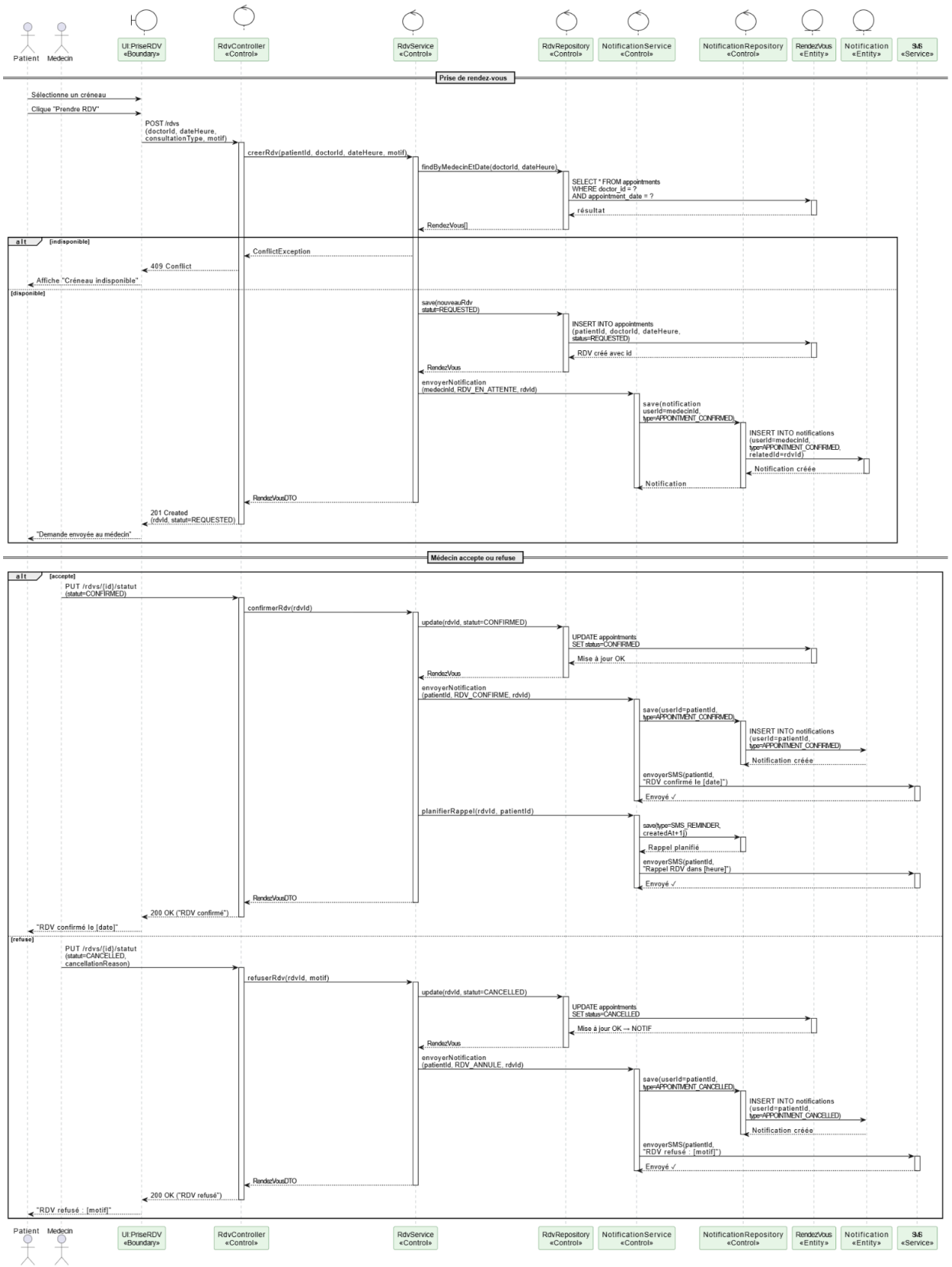


Figure 3.21 – Diagramme de séquence conceptuel du cas « Prendre un rendez-vous »

Conception du cas "Gérer les rendez-vous"

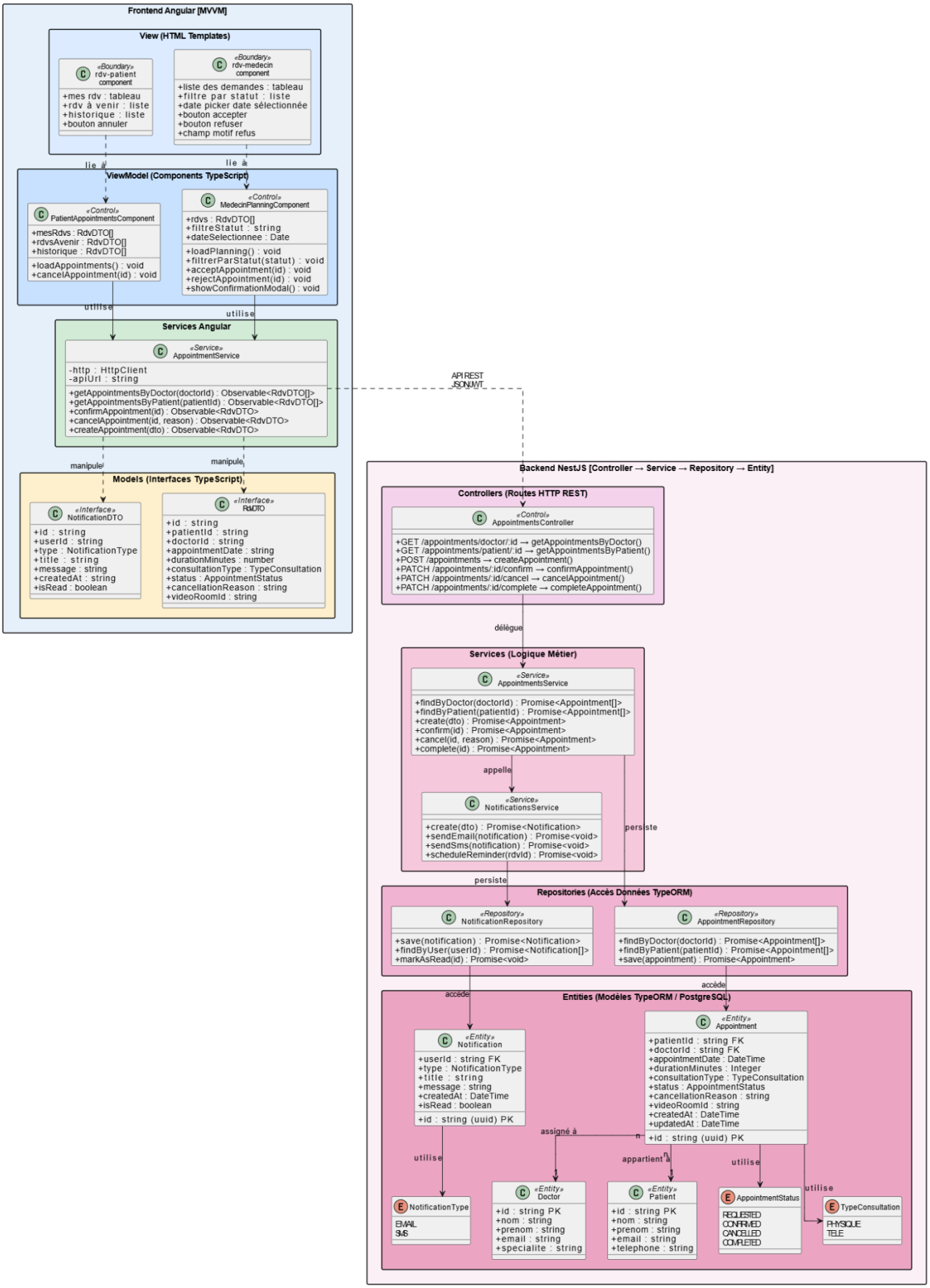


Figure 3.22 – Diagramme de classe conceptuel du cas « Gérer les rendez-vous »

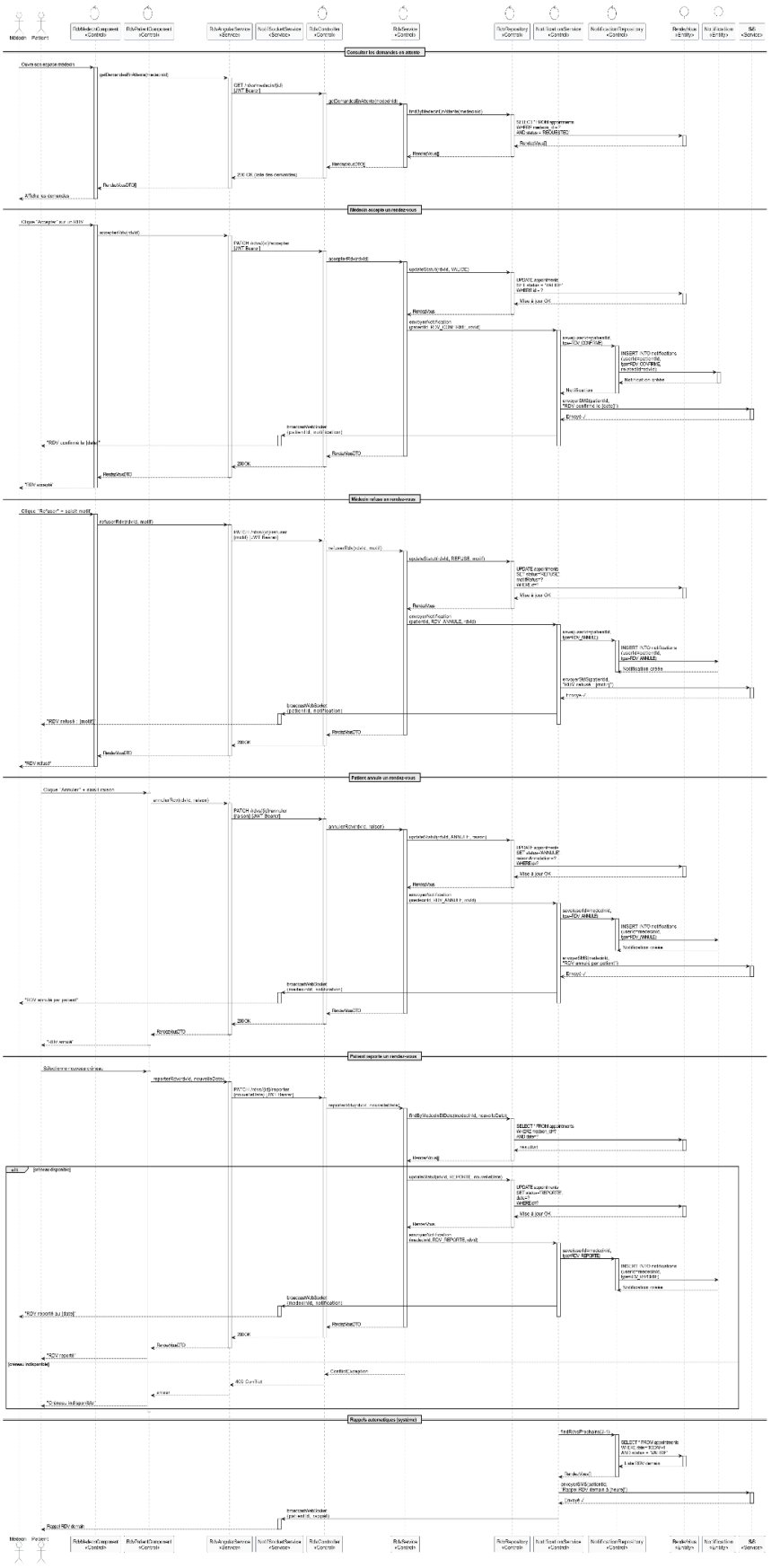


Figure 3.23 – Diagramme de séquence du cas « Gérer les rendez-vous »